CST1510 Programming for data communication and networks

Week 10: Implementing a GUI for the Shopping System

LAB ACTIVITY

In this lab, students will learn how to create a Graphical User Interface (GUI) for the shopping system using Python's Tkinter library https://docs.python.org/3/library/tkinter.messagebox.html. The goal is to provide students with a hands-on experience in building interactive interfaces that interact with the backend.

Step-by-Step Instructions

Part 1: Preparing Your Environment

1. **Objective**: Set up your project structure for GUI implementation.

2. **Steps**:

   A. Ensure Python (3.8 or above) is installed.
   B. Open your code editor (e.g., Visual Studio Code, PyCharm, etc.).
   C. In your project directory (e.g., `ShoppingSystem`).

3. **Prerequisites**:

   - A working client-server implementation of the shopping system.
   - Database schema initialized using the provided SQL file (`shopping_system.sql`).

Part 2: Creating the GUI Structure

1. **Objective**: Create the base GUI window using `Tkinter`.

2. **Steps**:

   A. Create a new file named `gui.py`.
   B. Import the `Tkinter` library:
   ```
   import tkinter as tk
   from tkinter import messagebox
   ```
   C. Create a main function for your GUI:

```
def setup_gui(client_functions):
    root = tk.Tk()
    root.title("Shopping System")
    return root
```

3. **Result**: When you run `gui.py`, an empty window should appear.

Part 3: Adding Input Fields

1. **Objective**: Add fields for `Product ID` and `Quantity`.

2. **Steps**:

   A. Create a frame for input fields:
   ```
   frame_inputs = tk.Frame(root)
   frame_inputs.pack(pady=10)
   ```
   B. Add labels and entry widgets for `Product ID` and `Quantity`:
   ```
   lbl_product_id = tk.Label(frame_inputs, text="Product ID:")
   lbl_product_id.grid(row=0, column=0, padx=5, pady=5)
   entry_product_id = tk.Entry(frame_inputs)
   entry_product_id.grid(row=0, column=1, padx=5, pady=5)

   lbl_quantity = tk.Label(frame_inputs, text="Quantity:")
   lbl_quantity.grid(row=1, column=0, padx=5, pady=5)
   entry_quantity = tk.Entry(frame_inputs)
   entry_quantity.grid(row=1, column=1, padx=5, pady=5)
   ```
3. **Result**: The window now displays fields where users can input `Product ID` and `Quantity`.

Part 4: Adding a Display Area

1. **Objective**: Create a `Text` widget to display results.

2. **Steps**:

   A. Add the `Text` widget below the input fields:
   ```
   text_display = tk.Text(root, height=15, width=60)
   text_display.pack(pady=10)
   ```
3. **Result**: A large text area appears in the window for displaying information.

Part 5: Adding Buttons for Actions

1. **Objective**: Add buttons to perform actions such as viewing products, adding to the cart, and checking out.

2. **Steps**:

   A. Create buttons and link them to placeholder functions:
   ```
   btn_view_products = tk.Button(root, text="View Products", command=lambda: print("View Products clicked"))
   ```

```
btn_view_products.pack(pady=5)

btn_add_to_cart = tk.Button(root, text="Add to Cart",
command=lambda: print("Add to Cart clicked"))
btn_add_to_cart.pack(pady=5)

btn_view_cart = tk.Button(root, text="View Cart",
command=lambda: print("View Cart clicked"))
btn_view_cart.pack(pady=5)

btn_checkout = tk.Button(root, text="Checkout",
command=lambda: print("Checkout clicked"))
btn_checkout.pack(pady=5)
```

3. **Result**: Buttons appear in the GUI, but they currently print placeholder messages.

Part 6: Integrating Backend Functions

1. **Objective**: Link the buttons to actual backend functions.

2. **Steps**:

   A. Replace the placeholder commands with client function calls:

```
def view_products():
    response = client_functions['view_products']()
    text_display.delete(1.0, tk.END)
    text_display.insert(tk.END, response)

def add_to_cart():
    product_id = entry_product_id.get()
    quantity = entry_quantity.get()

    if not product_id or not quantity.isdigit():
        messagebox.showerror("Error", "Please provide
valid inputs.")
        return

    response = client_functions['add_to_cart'](product_id,
int(quantity))
    messagebox.showinfo("Cart", response)

def view_cart():
    response = client_functions['view_cart']()
    text_display.delete(1.0, tk.END)
    text_display.insert(tk.END, response)

def checkout():
    response = client_functions['checkout']()
    text_display.delete(1.0, tk.END)
    text_display.insert(tk.END, response)
```

3. **Result**: Buttons now trigger actual backend functions and display results in the text area.

Part 7: Running the GUI

1. **Objective**: Integrate the GUI into the client and test.

2. **Steps**:

   A. Update `client.py` to initialize the GUI:

   ```
   from gui import setup_gui

   client_functions = {
       "view_products": get_product,
       "add_to_cart": add_to_cart,
       "view_cart": view_cart,
       "checkout": checkout,
   }

   root = setup_gui(client_functions)
   root.mainloop()
   ```

   B. Run `client.py` to launch the GUI.

3. **Result**: The GUI interacts with the server, displaying products, adding items to the cart, and performing checkout operations.

Part 8: Extending the GUI

1. **Objective**: Add additional features like clearing input fields after adding to the cart.

2. **Steps**:

   A. Add a function to clear input fields:

   ```
   def clear_inputs():
       entry_product_id.delete(0, tk.END)
       entry_quantity.delete(0, tk.END)
   ```

   B. Call `clear_inputs` in the `add_to_cart` function:

   ```
   def add_to_cart():
       product_id = entry_product_id.get()
       quantity = entry_quantity.get()

       if not product_id or not quantity.isdigit():
           messagebox.showerror("Error", "Please provide
   valid inputs.")
           return

       response = client_functions['add_to_cart'](product_id,
   int(quantity))
       messagebox.showinfo("Cart", response)
       clear_inputs()
   ```

3. **Result**: Input fields are cleared automatically after adding to the cart.

Lab Summary

By the end of this lab, students will have:

1. Created a GUI with input fields, buttons, and display areas.
2. Integrated the GUI with the backend client functions.
3. Tested the full functionality of the shopping system via the GUI.

In [ ]: